NTI Gymnasiet
Teknikprogrammet
Gymnasiearbete 100 p
HT 2023-VT 2024

Human-Bot Pair Programming

Software Architecting in Collaboration between Human and Bot using ChatGPT

Wilhelm Lidblad Jerrestål
Handledare: Jakob Envall
Examinator: Tuomas Mattson

Abstract

This study explores architecting software while engaged in a pair programming collaboration with ChatGPT. Its purpose is to assess the usability of ChatGPT as a collaborative partner to architect software. Its methodology involved a continuous informal conversation between humans and bots to create a code generator based on a web application for a specialized programming language used in Minecraft data packs. The study also investigates the effectiveness of this collaborative approach discussing factors such as code quality, productivity, and the ability to address ideas and requirements to ChatGPT. The study also brings up limitations to human-bot pair programming and solutions connected to the method as well as offering insights into the feasibility and potential benefits of integrating ChatGPT into pair programming scenarios, while also suggesting future research directions to further explore in regards to human-bot pair programming.

Sammanfattning

Denna studie utforskar parprogrammering tillsammans ChatGPT inom mjukvaruutveckling. Dess syfte är att utse möjligheten att använda ChatGPT som en programmerings partner för att utveckla mjukvara. Studiens metodik involverar att kontinuerligt konversera informellt med ChatGPT för att skapa en kodgenerator baserad på en hemsida för ett speciellt programmeringsspråk använd i Minecraft data packs. Studien undersöker också effektiviteten av denna typ av parprogrammering genom att tillgodose faktorer som kod kvalite, produktivitet och möjligheten att föra över ideer och krav till ChatGPT och få kvalitativa svar. Studien tar också upp begränsningar till människa - bot parprogrammering och lösningar kopplade till metoden. Den ger också insikt till hur möjligt och vilken potentiella fördelar av att integrera Chat GPT i parprogrammering med människor ger, men också föreslår framtida forskning för utforska människa - bot parprogrammering ytterligare.

Contents

1 Introduction

In recent years software architecting bots from different sources and companies have been developed to be able to create code. Other more general-purpose Artificial Intelligence such as Large Language Models (LLM), have shown to be capable of both code analysis and code architecting. These LLMs make it possible for ideas and code to be explained and conversed in written or spoken language, where responses can be explained and understandable. This makes it possible for humans to collaborate with LLMs like ChatGPT in informal language to architect software such as programs, websites, and servers. This can be a repetitive and challenging task, where experienced and Junior developers will encounter difficulties or spend time writing code whose logic is already planned. Challenges, however, can be made easier if developers were architecting together with large language models using methods like pair programming. Work can thereby be distributed and difficulties or encountered problems can be discussed and solved together as “partners”.

2 Purpose

The purpose of this case study is to examine if there is a possibility of using ChatGPT as a pair programming partner. This will be done by using informal language to architect a website code generator for a lesser-known and poorly documented programming language such as the one used in Minecraft data packs.

The main questions of this study are (1) how usable is ChatGPT when collaborating with developers in an informal scenario, to support and create a finished and usable product? (2) Document potential barriers to using ChatGPT in collaboration with developers, but also reflect on why these barriers exist and potential solutions.

2.1 Hypothesis

The hypothesis for pair programming with bots is that bots have a better capability of generating frontend HTML, CSS, and Javascript, but a lower capability of creating Minecraft data packs. Humans will then have a chance to contribute with knowledge that ChatGPT as of now gets wrong. The hypothesis was created on guesses that ChatGPT’s internet-based dataset probably contains a lot more information about website architecting than Minecraft data packs.

3 Background

3.1 Pair Programming

Software architecture has its methods for software engineers to produce, analyze, and learn. One method called Pair Programming enhances the quality of the code, code analysis, and the learning process (Haider & Ali 2011, Page. 21-25). By letting software developers work together in small teams, often situated around one workstation or by a remote connection to an Integrated Development Environment (IDE), different developers will contribute with different prior knowledge and experience but also use the given information about a specific problem in different ways (Wikipedia Pair Programming 2024, Design quality, paragraph 1). Bringing forth a larger number of methods to solve a problem, than a single programmer alone might do. As developers try to compensate for their differences, more ideas are brought up in conversation. This increases the design quality of the software as it lowers the chances of selecting a poor method (Wikipedia Pair Programming 2024, Design quality, paragraph 1).

When developers can converse general and specific code structures, they can prevent mistakes as they are made. Due to the collaboration between developers, a mistake made by an unaware developer is more likely to be seen and fixed by others, creating greater ability for developers to analyze code, besides also creating higher quality code. This makes the development process rather effective as both the design software and its code are of higher quality than what a single developer might have done. However, the effectiveness of pair programming might be limited to tasks where developers do not fully know how to complete them. Challenging tasks that call for creativity from the developers, give greater results if conducted in a pair programming scenario, whereas single software architects might be more effective in developing a specific software (Wikipedia Pair Programming 2024, Design quality, paragraph 1).

3.2 Large Language Models

3.2.1 ChatGPT

Chat Generative Pre-trained Transformer (ChatGPT) is a Large Language Model (LLM) using neural networks to follow the statistical context and placement of words in sentences. LLMs like ChatGPT can achieve general-purpose language “understanding” and generation by learning statistical relationships from training data (Nvidia n.d. What are Large Language Models?, Paragraphs. 1-6), during an intensive training process (Radford m.fl. 2018, Framework, Paragraphs. 1-5). ChatGPT gets its wide range of applications in natural language processing, psychology, and linguistics from the large dataset based on sources from the internet like Wikipedia, books, and publicly available data (Johri 2023, Paragraphs. 1-3).

ChatGPT is an example of a bot that can engage with humans to produce well-written responses to complex prompts (OpenAI 2022, Samples, Samples. 1-4). ChatGPT is not trained for software architecture, however, its ability to achieve general-purpose makes it very capable of understanding, analyzing, and responding to complex code. Its ability to provide explanations and engage in conversation creates the opportunity where users can refine and modify their code to meet a background goal without engaging in coding (Ahmad m.fl. 2023, s. 280).

3.2.2 Prompt Engineering

The process of prompting a query, command, or instruction with a specific style is called prompt engineering (Wikipedia Prompt engineering 2024. Paragraphs. 1-3). A response from an LLM such as ChatGPT may result in information being hallucinated. Hallucination is a response generated by an AI that contains false facts or misinformation (Wikipedia Hallucination (artificial intelligence) 2024. In natural language processing, Paragraph 1). Hallucinations are caused by a divergence in the data of the AI about the given task, for example, if an AI is prompted for information outside of its source dataset. It will attempt to fulfill the prompt with the provided information, resulting in made-up facts and faults. The cause can also be the model learned the wrong correlation between words (Wikipedia Hallucination (artificial intelligence) 2024. Causes, Paragraph 1-3). To minimize hallucinations and confusion in responses the prompt can be written so it expresses and emphasizes the specified task, to get the model to respond in a specific way or style based on the human's intention. Giving extra clear instructions or extra context can help, however

adding more information can also negatively impact responses and get the AI off course. AI could with additional information focus on less important parts of the prompt instead of focusing on the core task described.

3.3 Minecraft Data Packs

Minecraft data packs is a system in Minecraft Java Edition that can override or add to gameplay systems without modifying any game code. Users can combine modifications with combinations of built-in commands packed into functions to customize their Minecraft Experience. This creates a Data Pack that all users can create or add into game worlds to customize their in-game experience without the need for further installations of third-party programs (Minecraft Wiki. Data pack 2023. Paragraph 1).

Commands are advanced features that modify in-game behaviors and conditions set by the developers. Commands do not have to be run by data pack and can be executed through the console, chat, or in-game command blocks. Commands are limited to the scope of the game set by the developers. Commands can not add new in-game mechanics, only modify existing mechanics within the set limits. By combining commands into functions placed into a data pack, a script-based programming language reveals itself (Minecraft Wiki. Commands 2024. Paragraph 1).

4 Method and Material

4.1 Method

To answer the main question, a method of execution is needed. This method needed to take into account how the execution of the project actually will answer the main question. This calls for a project that poses sufficient difficulty for both humans and bots to execute but is easy enough to understand, an environment where collaboration is of great importance to further progress in the project. The project was split into two parts, frontend, and backend, based on the hypothesis explained in 2.1 to separate frontend HTML, CSS, and Javascript from backend javascript and data packs.

To create this project a method inspired by the study “Towards Human-Bot Collaborative Software Architecting with ChatGPT” (Ahmad m.fl. 2023, s. 280) was used. The method starts by creating an architecting story. This includes creating the overarching goal of the software, its requirements, and vague terms of how certain functions should work. The

architecting story was intentionally left undetailed to later challenge ChatGPT to adapt code structures to new ideas as the project went along.

The story was then given to ChatGPT as a prompt formulated as “create this”. The method then builds on a continuous conversation between the developer and ChatGPT in informal language as new prompts to further develop the software. New solutions created by ChatGPT were continually tested by the developer. Potential problems or errors with the solution were then returned to ChatGPT as feedback by the developer. Solutions could also be added by the developer and conversed with the bot for analysis. The method emphasizes collaboration and discussion with informal shorter prompts rather than longer formal prompts, while also letting both human and bot developers express their ideas and solutions. This exists to closely imitate the natural collaboration between humans when solving tasks in, for example, a pair programming scenario.

4.2 Material

- Account at OpenAI to access ChatGPT (GPT-3.5)
- Minecraft Account
- IDE, Visual Studio Code (VS Code)
- Github: To host website
- Tools and VS Code Extensions: To effectively develop data packs and websites
    (1) MCStacker
    (2) Misode Data Pack Generators
    (3) Live Server v5.7.9
    (4) Datapack Helper Plus v3.4.19
    (5) Syntax-mcfunction v0.6.0

5 Results

The results of this study are split into four parts, the resulting architecting story, the resulting web application, and example prompts and problems. The Architectural story describes the initial idea and plan while the resulting web application shows the results of three months of development.

5.1 Resulting Architecting Story

Part one web application. A web application/website using html, css and javascript. In the middle is a 3 x 9 grid looking like a minecraft inventory for a single chest. Clicking on a square in the grid should open a menu on the left. Menu will contain five options. First the type of item, selected from a list of items using a dropdown menu that sorts from a json array with all items, using an input from the user. Secondly, I want an input field for the name of the item. And thirdly i want an input field for the function that should be run (note that function is not javascript just a metaphor for later files.) The program should then condense each slot or square in the grid with its following item type, name and function(if given) to a json array, start with slot 0 in the top left and go right reaching slot 8 and then go back to middle left. I want you to generate one file for html, I don't want any menus or grids to be generated here. One css file to set where the grid and the menus should be. As many javascript files as needed, preferably so that it's easy to read the code. Also create a json file with some test items from the game of minecraft. The javascript code should then generate the grid and menus, let the user click the slots in the grid, and open a menu to change options for that slot. And then after each change print out a json array with each slot and its corresponding options, If one slot is empty, skip that slot, so json should only contain used slots. Generate complete code for all files.

5.2 Resulting Web Application

The result of this study was a web application based on the architecting story and the development process described in the method. It was created with 136 documented prompts to ChatGPT and around 20-50 undocumented prompts over a timeframe of three months.

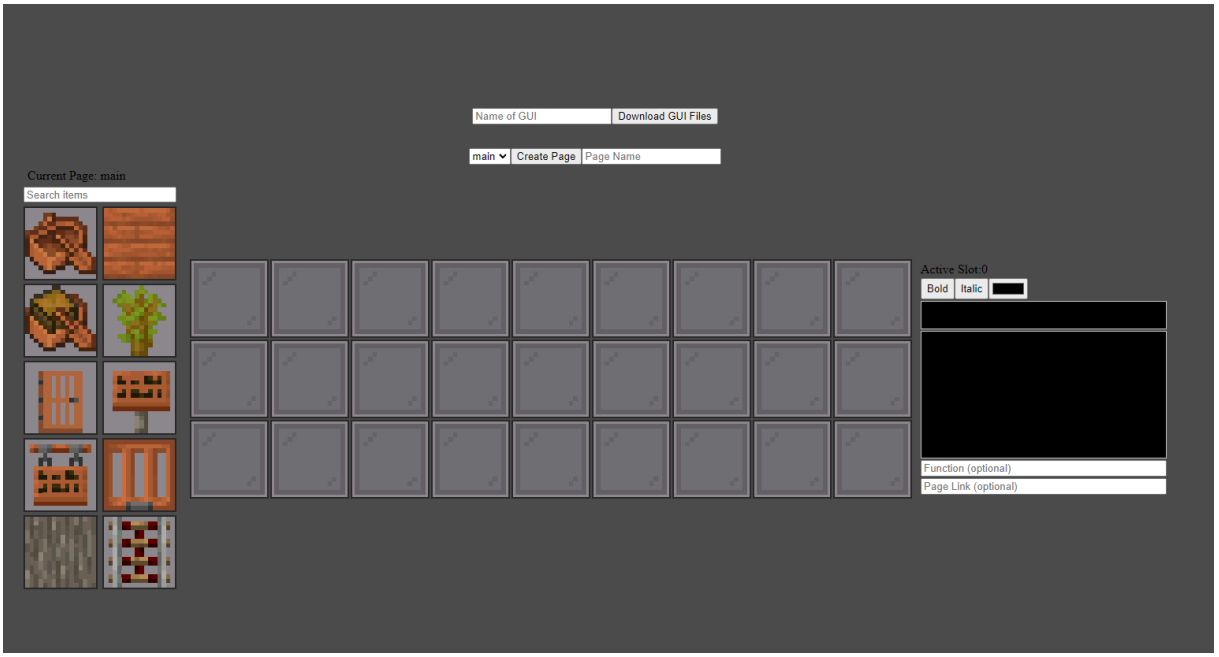

Image 1: An image of the user interface for the Code Generator Web Application.

5.3 Example Prompts

Example prompts are taken from the prompts list in the attachments. These were selected by quickly scanning through and selecting ones that seemed interesting, aiming to highlight diverse and engaging interactions.

Prompt 12

12 (A bundle of prompts as they are short and more debugging)

Today I would like to build further on user availability. I would like to have a new grid or list of slots where there is a complete list of all items. I then want the user to be able to drag any item from that slot and place it into any slot on the inventory grid. WHen releasing the slots, the item and icon should update. The user should still be able to manually change the item in the item menu. Write the js code to generate the new grid and the functionality to drag and drop items icons.

4 you say swap, i don't want to swap i want to copy only the icon and the item type no name no lore no function

Nothing is happening when dropping.

It shows that it does no allow me to drop the icon at another slot

Now it allows me to drop but it does not update the icon or the item type

Uncaught TypeError: Cannot read properties of undefined (reading 'icon') at HTMLDivElement.<anonymous> (main.js:216:62)

Same error

Gives no error but does not update anything

Still not working, i see the dragged item icon but nothing updates when its released over a slot

Prompt 17

17

I would like to make the item list not change or move the rest of the website if there is less or none items after searthing. SO the box is always the same size

Prompt 20

20

Okay now i we got this function:

function generateJSON() {

}

I now want you to create a new js file that contains the backend, That takes the json input and turns it to code following my instructions. but before that i want to disscuss some code ides.

In minecraft datapack, things that need to work.

A .mcfunction functipon that kills the old gui, summuns a new gui with the items in it.

A file that checks if a player has picked up an item from the chest minecart. if then remove the item, run the function from the item if there is one. And replace the items in the minecart. This should work for any player interacting with the gui. Can you create a rought outline of how this code in datapack would work. And then after i have looked at it we can see how we can make this from javascript

Prompt 21

21

Lets do the first step. Writing a javascripts function that generates a .mcfunction file called init.mcfunction. In this file there should be code to summon a chest minecraft with the items and name, lore and a custom nbt tag for each item with the fuction if there is one, for each item. I will give an example of how the syntax for summuning the chest is. example: summon chest_minecraft ~ ~ ~ {Items:[{Slot:0b,id:"minecraft:stone",Count:1b,tag:{display:{Name:'{"text":"name","color":"white"}',Lore:['{"text":lore","color":"white"}']},function.["test"]}}]}

Prompt 101

101

The generator creates the correct name as that works but the lore is not displayed so there is an error and not correctly formated. i will give you and example of how it should be. Update the code for lore then
stone{display:{Name:'[{"text":"hej","italic":false,"color":"yellow"}]',Lore:['[{"text":"ob","italic":false,"color":"red"},{"text":"a","italic":true},{"text":"ma was he"},{"text":"re","bold":true}]','[{"text":"","italic":false,"color":"red","bold":true}]','[{"text":"nub","italic":false,"color":"red","bold":true}]','[{"text":"","italic":false,"color":"red","bold":true}]','[{"text":"ri","italic":false,"color":"red","bold":true},{"text":"g","color":"gold"},{"text":"hth"},{"text":"ere","italic":true}]']}} 1

Prompt 102

102

Found the problem:

Lore:['[{"text":"nub","color":"#FF0000","italic":false},{"text":"nub","color":"white"}]','{"text":""}','[{"text":"hel","color":"white","bold":true}]']}}}}]}

each line has to have ' around them.

Prompt 103

103

it worked. Next update is to be able to add varibles to item name and lore. Making then dynamicly display something instead of being contant. You will not be able to change the color or style of the varible but it will be in text and create its own object.

i varible should be added by typing $(varible name) inside of eaither name or lore input.

```
// Function to extract name segments from input
function nameSegmentsFromInput(htmlString) {

}
```

this function should make it its own object. normal text has it ike this

```
text: text,
      color: colorStyles[index],
      bold: boldStyles[index],
      italic: italicStyles[index],
```

but i just want the {$(varible name)} to be its own object. This makes it easy to in minecraft using datapacks input entier object with names and color and italic and bold. Making it even more powerfull.

updatew the code abouve to support my idea. But keep all the current functionalitys. Dont want to mess with that.

Prompt 126

126

Okey this works fine. I now want to add a new big functionality. This should work with the previeos functionalitys. Just some reworking is needed. The new functionailty is to add pages. A page is a list of slots like the JsonData right now. Adding more pages means that jsonData need to be expanded to have pages and for each page have 27slots. This is so the user can have more pages in the guj and switch between the. the first step is to update the code to allow for pages in the jsonData varible. This means that the old functions should be able to write and save and load, but with a new varible called page. So if im on the page page1 then it should write and save to page1 if i thgen switch or create a new page it should write to that page object in the json.

I would the the pags to work like this. When the website is started we have a defualt page. Everything i ahve up to this point should work here. the only diffrence is that jsonData should show Pages[
main{
Slots[
]}]
So like before if i edit and item the items slot and name and lore and stuff should update in pages main slots the slot and name / lore. I now want to add the functionality that a user can add a new page with a choosen name. SO it becomes like this in the json

Pages[
main{
Slots[
]},
page1{
slots[
]}]
Now the active page is page1 this should have a clean inventory grid with the defualkts. editing items here should now update main.slots but page1.slots. Then there should be the ability to switch between pages. A switch means that it saves the pages items and the setting for those slots/items and loads the other items. This shoiws the items and its corresponding setting on the website.

---

you should be able to create this or update the code from the javascript code i gave you down below. Look at how the code works and try to rework the functionality now to also work with the pages. dont include jsonOUtput that is something old insted console.log(jsonData). Write all the code that is also inbetween the new, so if there are 4 new function from line 1 then you should also include the old code so i can just copy paste.

5.4 Example Problem

Problem encountered during the development process to convert HTML string to formatted JSON.

(1) HTML String:

HTML

(2) Formatted HTML code:

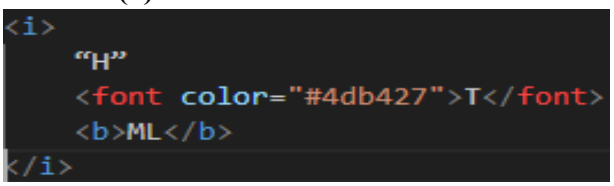

Image 2: HTML string formatted as HTML code.

(3) Formatted JSON Data:

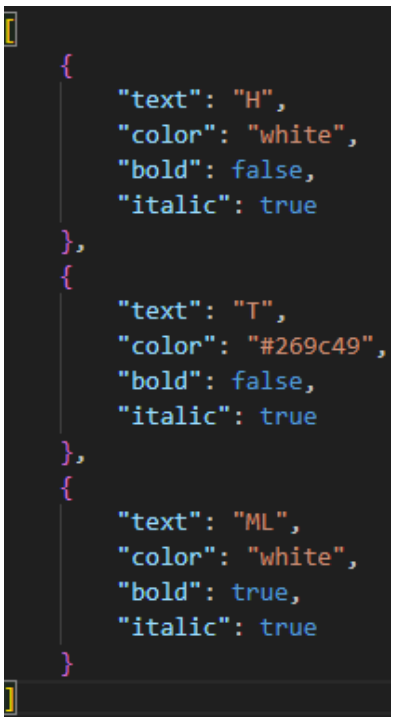

Image 3: HTML string formatted as JSON data.

(4) Displayed HTML String:

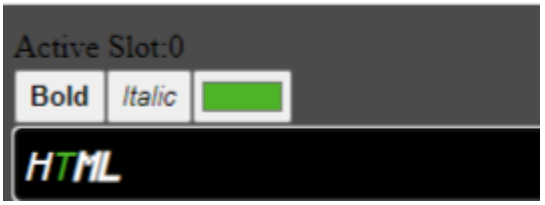

Image 4: HTML string displayed on the web application as styled text.

6 Discussion

Note that the results presented do not include responses and resulting code from ChatGPT. This is motivated due to the conversation being active and long and responses including complete files of code with small changes. Prompts were left vague, informal, and misspelled to experiment with ChatGPT's knowledge and understanding of informal language as conversations between partners in collaborations could be informal. But also to display pair programming scenarios where solutions and information are not presented to the participants with formal language or with deep explanations of the problem such as error messages.

6.1 Human vs Bot

This study covers a particularly new subject of using Large Language Models to architect software alongside human developers. Creating the relevance to discuss whether human-bot pair programming is necessary, or if it is instead being overshadowed by human-human pair programming or bots architecting software completely autonomous.

The results of this study show that tools like ChatGPT are not capable of architecting software completely autonomously because there will (for now) exist holes in the training data sets for these models. For example, the ability to architect working and up-to-date code for data packs, instead of guessing or responding with outdated code is not an ability that these models currently have. In these situations, a human developer is the only one capable of architecting the software required.

Results from this study show that even in situations where ChatGPT is capable of architecting code it cannot sometimes add to or update code to accomplish new goals or requirements (Attachment 4). This requires human competence and problem-solving, an ability Large Language Models do not yet have and may never achieve. Human-bot pair programming is necessary as current Large Language Models do not have the capability of architecting autonomously.

Humans on the other hand are very capable of autonomously architecting software. Using pair programming as a method for software architecting to produce, analyze, and learn a group of humans can effectively and meaningfully architect software with the positive effect mentioned in 3.1. The difference between human pair programming and human-bot pair programming is the immense library of architecting the bot can support humans with, making it hard to compare with other methods such as human-human pair programming (Sarkar 2022.

Page. 128). Humans have the ideas, the problem-solving skills, and the knowledge to begin architecting software. Whereas results (Attachments 3 & 4) show that Large Language Models for instance ChatGPT can quickly architect code utilizing best practices, known methods of execution, performance differences, error handling, code analysis, code refactoring, and commenting.

Differences between humans and bots create good scenarios for pair programming, supplementing each other to architect software, and giving way for rapid software development that's both effective and minimizes mistakes, security concerns, and poor code structure (Wikipedia Pair Programming 2024, Design quality, paragraph 1). Responses from ChatGPT even for complex and large code are returned to the human developer within minutes. It's reasonable to state that human-bot collaboration architects software faster than human-human collaborations. In this study, it lowered the completion time for a project, lowered mental resources spent by the humans on code architecting, increased productivity relative to time spent, and created possibilities for additional functionalities. It could also increase enjoyment and motivation in the project alongside the notable progress made by the collaboration. The software architected during this study did not however aim towards best practices or code readability, concepts that could be further developed and studied in regards to pair programming between humans and bots. This concludes that human-bot pair programming is a relevant topic necessary at this moment in time, where tools like ChatGPT can bring huge value to how developers architect well-written code, but also support human ideas and solutions, before a time where models may be capable of autonomously developing software.

6.2 Pair Programming between ChatGPT And Humans

This study takes a more humanized approach to large language models where tools like ChatGPT are thought of more as a person instead of an instruction-based software. This is important when evaluating pair programming between humans and ChatGPT and discussing its differences from pair programming between humans, as the nature of collaboration is related to humans. Results from this study would suggest that pair programming between humans and ChatGPT has the same effects as human pair programming with some key differences. A humanized approach creates the core concepts for the method used in this study, where the conversation is highlighted as more important than the resulting response to a prompt. This is why the method takes a more lenient approach to prompt engineering, where prompts are informal, short, and nonspecific. Even if the resulting prompt may set ChatGPT off in the wrong direction, the human developer always can correct and help during the active conversation.

It may be that humanizing large language models is the wrong thing to do as it's more a program than a human. This approach to pair programming with tools such as ChatGPT is discussed more in 6.2.3.

6.2.1 Overview of the Project

Overviewing the architected software dedicated to this study reveals that pair programming using ChatGPT is feasible to a limited extent. The result (Results 5.2 & Attachment 1-2) of this study indicates that it is feasible to use ChatGPT in an informal pair programming collaboration to architect a code generator for the poorly documented programming language used in Minecraft data packs. However, ChatGPT limitations discussed in 6.1 conclude that the project has to be a collaboration between humans and bots.

The project created for this study is a GUI generator that creates an in-game graphical interface within a Minecraft chest minecart. Based on a web application using Github Pages the generator was designed to test ChatGPT's ability to work with different code structures and work with a poorly documented programming language. The software architecture was designed on the principle that Minecraft’s in-game storage containers such as chests, barrels, and chest minecarts could be modified with data packs to act like a GUI. The process of creating these GUIs is repetitive and could therefore be created using a generator. The web application shown in results 5.2 was finished within a reasonable time frame, where the time designated was approximately four months and the project used three months to be

completed. The web application included all the features stated in the architecting story, but also some features like different pages (menus) and dynamic variables that were added as the project went along. These ideas were not planned to begin with, due to reasons stated in the 4.1. They were incorporated into the final project to test ChatGPT's ability to refactor old code to fit with new goals and requirements. A big question the study brought up during the architecting process is how much adding, changing, or removing features complicates collaboration with ChatGPT compared to thoroughly planned projects. Based on this project alone it would seem that changes to planned projects are inevitable and even that it makes the final project more robust, efficient, or better aligned with the needs of its user. The difference may be that pair programming using ChatGPT yields better results due to the extensive power that Large Language Models have compared to humans.

Evaluation of the project would determine it as a successful and complete project in the eyes of this study, where it has fulfilled its purpose to examine the usability of ChatGPT in a pair programming collaboration with humans. It also shows the usefulness of the method to architect software for this particular project and hints at being a general method for developing software while further research is needed in this field to verify these claims.

6.2.2 Pair Programming With ChatGPT

Software architecture demands polished and thoroughly planned code structures. Methods to effectively and with greater quality develop software such as the ones pair programming brought up in 3.1, are great ways of developing software. 6.1 discussed differences between the pair programming techniques, human-human, and human-bot, however, it does not bring forth that solo human developers have the opportunity to enhance their development by fully or partly switching to human-bot collaboration.

The study's methodology emphasizes conversion over strict prompt engineering, and results prove it to be effective in productive software architecting. As discussed earlier it shows that collaboration with ChatGPT in a pair programming scenario is feasible, however, assessing how usable tools similar to ChatGPT are to develop software compared to more traditional human architecture is harder to conclude. This is due to factors such as ChatGPT hallucinating information that it can not accurately use or give due to reasons like, training data limitation, complexity of task, and formulation of the prompt which will be discussed further in 6.3. These limitations, results from this study show, can be overcome by continuously conversing ideas and problems in the text back to ChatGPT as prompts.

This is shown in the 5.3, the human expresses its idea in example prompt 101, ChatGPT then tries to fix the problem with a response but the problem persists, and the human returns with example prompt 102 to fix the problem with knowledge of the source problem. This back-and-forth conversation between the bot and the human is the driving force behind the progress of the project. The data pack part of the project was instead created by the human writing all code separately, then using ChatGPT the data pack files were incorporated into the javascript code, to be generated as files based on inputs from the web application. Here both parties are equally as important to the project as ChatGPT could never have written the data pack code needed and the human could not in the same timeframe write the javascript code. Without the collaboration the project would either be abandoned or would have accumulated more time spent, this is due to a human developer being able to develop and incorporate data pack features alone but while doing that it also lowers the effectiveness of the project as discussed in 6.1. The human developer can, when faced with limitations from ChatGPT, work around the limitations by manually writing code to then converse with ChatGPT. This suggests that the usefulness of large language models in pair programming is directly connected to the skill level of the developer and increases as the knowledge and experience of the developer increases.

By this, the usability of ChatGPT in a pair programming scenario is affected by the knowledge of the human developer. The split in the structure described in 4.1 was only possible due to humans understanding how Minecraft data packs work, otherwise this project would not have been possible to conduct. Pair programming with ChatGPT is also limited by the knowledge of the human, making this method less applicable for less experienced developers. Problem-solving skills also become useful when understanding code functionality and code behavior is not needed in a collaboration with ChatGPT. ChatGPT is not capable of solving new problems that are not part of its training data alone without human support. A developer is not required to know the inner workings of the subject that ChatGPT can support, however, the developer is fully required to understand a problem when working with information that ChatGPT tends to hallucinate. This suggests that complexity is also a factor in the usability of ChatGPT as the complexity goes up the usability goes down and this is further explained in 6.3.1.

This study does not prove this is always the case for human-bot pair programming but it hints at the methodology being a good driving force for software development. It also indicated that new tools like ChatGPT can be harnessed with methods that provide the human developer

with greater possibilities to develop software based on design, functionality, and creativity instead of being limited by understanding different code structures, programming languages, and more.

6.2.3 Alternative Methods

The method explained in 4.1 sets up the ground principles for human-bot pair programming for this study in particular. Its purpose is to connect humans and bots to develop software together effectively, however, its purpose is also to act as a solution to limitations that will be discussed in 6.3. If alternative methods bring forth greater potential in human-bot collaborations, the results of this study do not show. However, even if a definite answer can not be made without future research discussed in 6.4, alternative methods to pair programming can at least be discussed. This study shows that in certain situations, prompt engineering would have been a better alternative to solve a given problem. An active conversation will in some cases not solve a given problem due to limitations in ChatGPT, therefore the approach seems to be to prompt engineer ChatGPT to respond as the developer needs.

For example cases where a developer knows exactly how a solution would be created, a prompt, engineered to architect that solution is of greater usefulness than a long conversation with the risk of limitations discussed in 6.3 to reach the same result. Lowering the risk of frustrated developers leads to faster workflows and lets developers instead focus on solving problems more than architecting the solutions manually. Prompt engineering may also be a better solution to architect software with fewer mistakes, bugs, and poor code execution on specifically bigger projects. Results showed that using active conversations on bigger tasks such as refactoring a file created room for ChatGPT to make mistakes with information explicitly not mentioned in the prompt, whereas smaller tasks had a greater focus from both ChatGPT and human developers. This indicates alternative methods may be more suitable for bigger projects or tasks while the methodology outlined in the study is more suitable for small-scale projects or tasks.

This suggests that prompt engineering can be a valid method when ChatGPT is tasked with bigger changes or add-ons to the code, however, smaller conversations like the method used for this study are better when architecting smaller projects or performing smaller tasks as part of a larger project. Prompt engineering as a method steers away from collaboration and more into automation also described in 6.1, autonomously architecting software is far off the method and focus of this study, where it instead focuses on collaboration between humans and bots.

6.3 Limitations

During the project, a few roadblocks appeared limiting the usefulness of ChatGPT in a pair programming scenario. These limitations were directly connected to the project and methodology for this study and do not show a complete understanding of human-bot pair programming limitations. One reason for this is that only one project was created to conduct this study where several diverse projects encountering different problems and solutions would get a better understanding of these limitations. However, the limitations discovered from the result are augmented to be general limits of human-bot pair programming. Complexity, Hallucinations, and Humans are all general limitations and would affect most projects that implement human-bot pair programming with the method explained in 4.1.

6.3.1 Complexity

As the complexity of tasks increases the need for problem-solving skills also increases and that complexity limits what tasks ChatGPT can effectively respond to. This can be caused by programming problems not being documented elsewhere or the problem being very niche or overly complex. As ChatGPT in principle does not have the ability to problem solve only act on data inside its dataset, this means a problem's complexity that limits ChatGPT, does not necessarily need to be complex to humans. Tasks that are harder but not complex to humans could seem complex to a large language model and would limit their ability to carry out the task needed.

An example of the complexity limitation in the software developed for this study is a function that translated an HTML string to a JSON format, see 5.4. HTML tags such as <i> for italic, and <b> for bold, enclose letters to display their styling (image 4), the complexity issue created from this lies in the combinations of tags that can be applied to style text. Each style

can be applied in a different order, resulting in a different HTML string with the same visual representation. This created the need for a function to be able to handle all different combinations of tags to ensure the correct translation to the JSON format. ChatGPT's software solutions continuously failed tests handling different combinations of tags, resulting in a long and frustrating conversation. Interestingly enough the AI could in response, correctly translate strings to JSON (Attachment 4.1). In this scenario, the method calls for a human intervention using its problem-solving skills to solve the programming problem at hand. In this case, an algorithm of searching the string from left to right, collecting the active tags, and starting a new JSON element when an end or starting tag is found was presented to ChatGPT in text. Immediately the response was of working conditions for all currently tested cases (Attachment 4.1). This example supports the method of humans and bots actively conversing ideas and problems to create and solve them but also supports the ideas discussed in 3.1 of pair programming's effects on software development. It also shows that complexity is a current limitation of large language models.

6.3.2 Hallucinations

As stated in 3.2.1 large language models get their usefulness by training them on large datasets, a problem explained further in 3.2.2 about hallucinations and holes in the dataset creating a limitation on LLMs usefulness. Information that is not within the dataset is a huge limitation to how useful ChatGPT can be in pair programming. Information that is not part of the training dataset will not be generated accurately, thus creating a barrier to what is possible without human support. As explained in 6.2.3 alternative methods such as prompt engineering reach a limitation here compared to the methodology used in this study. An example of this is shown in the result is the data pack part of the project. It proved the hypothesis explained in 2.1 and showed that limitations in the ChatGPT dataset limited the usefulness of the model to perform software architecture, instead relying on human support discussed in 6.2. The methodology used aims to solve or work around this limitation by introducing humans to support ChatGPT.

Smaller limitations also exist like when ChatGPT gets confused or acts on the wrong information causing a response of code not to work properly. ChatGPT also has other limitations when tasked with updating or refactoring code due to ChatGPT prioritizing information stated in the prompt and forgetting less important parts of code that also need to

get updated or rewritten. Here human support is needed as what could seem logical to a human that when changing one part of the code, you will need to update dependencies or code depending on the updated code, however, this is not always the case with large language models. This shows that hallucinations are a big limitation to letting ChatGPT develop code without human interference.

6.3.3 Humans

The study's methodology and prompt engineering both necessitate substantial software development knowledge from human developers. Prompt engineering suffers from this need for knowledge more so than an active conversation between humans and bots. Suggesting that it requires less knowledge to architect software using the method in this study (Sarkar 2022. Page. 143). This however also limits the usefulness of ChatGPT as a partner in pair programming where the limits of human problem-solving skill and creativity are prominent when pair programming with ChatGPT.

Human limitations also limit the usefulness of ChatGPT as discussed in 6.3.1 and 6.3.2 and are directly solved by humans intervening to help and support with solving problems. For software architecting this as a result increases the human skill needed to use ChatGPT effectively as it minimizes other limitations. 6.2.2 brought up that the usefulness is directly tied to the skill level of the human developer, however, it also seems that the limitations to using ChatGPT decline as the human skill level increases (Sarkar 2022. Page. 144).

6.4 Future Research

This study is a case study conducted with the result and perspective of a single individual. Therefore it does not contain any proof that methods or results are consistent for every individual or project in other cases. This study is conducted merely to bring up and discuss the possibilities and usefulness of human-bot pair programming while using the method used in this study. Future research is needed to conclude with more extensive and diverse proof that human-bot pair programming does work in a broader scale of tests, but also to see the usefulness of ChatGPT in architecting software.

Future research could also be focused on finding differences or concluding how or why alternative methods are better in pair programming with ChatGPT. For example, examine whether formal language is better to use than informal language even in active conversation. Research could also be focusing on finding evidence on the effects of supporting LLMs using human knowledge and how that may affect software quality and design choices, compared to human-human pair programming.

7 Conclusion

Pair Programming between humans in software development is a method that enhances code and prevents mistakes. Pair Programming collaboration between human programmers and ChatGPT, results from this study suggest is even greater at enhancing code structures, preventing mistakes, and overall architecting software faster and better than human-human pair programming

Large language models with their immense power to architect software in a short time frame with effective and great methods of execution make it possible and even useful to engage in human-bot pair programming collaboration. The usefulness seems to range from project to project depending on limitations like the complexity of the project, the dataset ChatGPT was trained on, and human knowledge. Results state that potential solutions to these limitations are a continuous conversation between humans and bots, to develop the software forwards together.

Source List

Ahmad, A., Waseem, M., Liang, P., Fahmideh, M., Aktar, M. S., & Mikkonen, T. (2023). Towards Human-Bot Collaborative Software Architecting with ChatGPT. Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '23), June 14–16, 2023, Oulu, Finland (s. 279–285). ACM, New York, NY, USA. https://doi.org/10.1145/3593434.3593468 (2023-11-4)

Haider, M. T., & Ali, I. (2011). Evaluation of the Effects of Pair Programming on Performance and Social Practices in Distributed Software Development (Master's thesis, Blekinge Institute of Technology, School of Computing). Retrieved from https://www.diva-portal.org/smash/get/diva2:832682/FULLTEXT01.pdf (2024-01-10)

Sarkar, A., Negreanu, C., Zorn, B., Ragavan, S. S., Poelitz, C., & Gordon, A. D. (2022). What is it like to program with artificial intelligence? In PPIG 2022 - 33rd Annual Workshop. Retrieved from https://www.ppig.org/papers/2022-ppig-33rd-sarkar/ (2024-01-17)

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI. Retrieved from https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf (2024-02-09)

OpenAI. (2022, November 30). ChatGPT: OpenAI's New Language Model. OpenAI Blog. [Blog]. Retrieved from https://openai.com/blog/ChatGPT (2024-02-01).

Johri, S., & Moncada-Reid, C. (2023, June 6). The Making of ChatGPT: From Data to Dialogue. Science in the News Blog. [Blog]. Retrieved from https://sitn.hms.harvard.edu/flash/2023/the-making-of-ChatGPT-from-data-to-dialogue/ (2024-02-09)

NVIDIA. Large Language Models. NVIDIA. Retrieved from https://www.nvidia.com/en-us/glossary/large-language-models/ (2024-01-19)

Minecraft Wiki. Data pack. 2023. Retrieved from https://minecraft.fandom.com/wiki/Data_pack (2024-01-18)

Minecraft Wiki. Commands. 2024. Retrieved from https://minecraft.fandom.com/wiki/Commands (2024-01-18)

Wikipedia. Pair programming, 23 January 2024. Retrieved from https://en.wikipedia.org/w/index.php?title=Pair_programming&oldid=1198323820 (2024-01-10).

Wikipedia. Prompt engineering, 28 February 2024. Retrieved from https://en.wikipedia.org/w/index.php?title=Prompt_engineering&oldid=1210775132 (2024-02-24).

Wikipedia. Hallucination (artificial intelligence), 22 February 2024. Retrieved from https://en.wikipedia.org/w/index.php?title=Hallucination_(artificial_intelligence)&oldid=1209562009 (2024-02-24).

Attachments

(1) https://github.com/wi1helm/Gymnasiearbete

(2) https://wi1helm.github.io/Gymnasiearbete/

(3) Prompts Document: “Prompt Attachments” (file)

(4) ChatGPT Conversations:

(1)https://chat.openai.com/share/b29d522a-8860-4d12-b000-98b62ced3131

(2)https://chat.openai.com/share/5df08eca-6c81-4484-a8cb-b0e279ffa20e